

LexGraph

Predicting Outcomes and Grounding Authorities in Indian Supreme Court Land-Dispute Judgments

A complete account for a reader starting from zero

CSE 573 — Semantic Web Mining — Arizona State University

1 October 2026

Abstract

This document explains, from first principles and assuming no prior knowledge of law or machine learning, what this project set out to do, what was built, what was measured, and — in equal detail — what turned out not to be true. It is written to be read straight through by someone who has never seen the repository.

The project began as a system that would read a court judgment, pull structured legal objects out of it, and reason over them. It became something more useful and less flattering: a carefully leakage-controlled measurement apparatus which shows that *85% of the apparent advantage of the published input construction is disposition language surviving the deletion of the operative order*, that the court’s reasoning itself adds only a small real amount, and that *converting facts into a symbolic vocabulary destroys most of the predictive signal they carry*. Both claims are supported by controlled experiments with confidence intervals, and both contradict what the project originally assumed. Along the way several genuinely positive results were established — statute prediction and precedent retrieval both work well above their baselines, and the system’s explanations were shown to be faithful rather than decorative.

Nothing in this document is an estimate or a projection. Every number is read from a results file produced by a script in this repository, and Section 10 states for each number whether it is current or awaiting recomputation.

Contents

1	How to read this document	3
2	The problem, in ordinary language	3
2.1	What a judgment is	3
2.2	The task	3
2.3	Why anyone cares	3
2.4	What was already published	4
2.5	The two questions	4

3	The one idea: leakage	4
3.1	An analogy	4
3.2	What must be removed, and why	5
3.3	The control that makes everything else interpretable	5
4	The data	6
4.1	The corpus	6
4.2	Where the labels come from	6
4.3	How the data is split, and why it is not random	7
5	What was built	8
5.1	Infrastructure: one door to the model	8
5.2	Segmentation	8
5.3	Fact extraction, with verbatim grounding	8
5.4	Claims, defences, elements	9
5.5	The canonical vocabulary	9
5.6	Claim families and fact patterns	9
5.7	Grounding facts in law: statutes and precedents	10
5.8	Outcome models and feature groups	10
5.9	Traces	10
6	Results	10
6.1	Headline 1: the published result is mostly leakage	10
6.2	Where the advantage actually comes from	12
6.3	Headline 2: turning facts into symbols destroys the signal	15
6.4	Where structured facts <i>do</i> pay off	16
6.5	Outcome prediction hits a ceiling, and the ceiling looks real	18
6.6	The explanations are faithful	19
7	What it took: the bugs that mattered	20
7.1	Four transferable lessons	21
8	Claims this project held and gave up	22
9	What is <i>not</i> established	22
9.1	Open blockers	23
9.2	Other open items	23
10	Status of every number in this document	24
11	Where the project stands	25
11.1	What exists and works	25
11.2	What the paper would say	25
11.3	What comes next, in priority order	26
12	Glossary	26
A	How to reproduce everything	27
B	Where things live	28

1 How to read this document

Section 2 explains the problem in ordinary language — what a court judgment is, what we are trying to predict, and why anyone would want to. Section 3 explains the single idea that governs every design decision in the project; if you read only one section, read that one. Sections 4 and 5 describe the data and the machinery. Section 6 reports every result. Section 7 is about the unglamorous work — the fifteen bugs that would each have produced a confident wrong answer. Section 8 lists the beliefs the project held and had to give up. Section 9 is the honest inventory of what is *not* shown. Section 12 is a glossary; feel free to jump there at any point.

A reader in a hurry should read Sections 3, 6.1 and 6.3, which contain the whole argument.

2 The problem, in ordinary language

2.1 What a judgment is

When a dispute reaches the Supreme Court of India, the court eventually publishes a document called a *judgment*. A typical land-dispute judgment in our corpus is around 5,300 words long and has a predictable shape, though almost never explicit headings:

- (1) a **caption**: the case number, the parties' names, the judges, the lawyers;
- (2) the **facts**: who owned what, who sold what to whom, when, and what went wrong;
- (3) the **procedural history**: which lower courts already heard this and what they decided;
- (4) the **arguments**: what each side's counsel submitted;
- (5) the **reasoning**: the court's own analysis — the statutes it reads, the earlier cases it follows or distinguishes, and the inferences it draws;
- (6) the **order**: the operative disposition. One or two sentences. "The appeal is allowed." "The special leave petition is dismissed with costs."

Item (6) is the outcome. Items (1)–(4) are, roughly, the inputs that existed before the court decided. Item (5) is written *afterwards*, by the winner of the internal argument, to justify a conclusion already reached. That asymmetry is the whole story of this project.

2.2 The task

Legal judgment prediction is the task of reading some portion of a case and predicting its outcome. In our formulation the label is binary:

- **WIN** — the appellant or petitioner succeeded (the appeal was allowed);
- **LOSE** — they did not (dismissed, rejected).

Cases that were sent back to a lower court (REMAND), succeeded in part (PARTIAL), or whose disposition our two independent labellers disagreed about (UNRESOLVED) are held out of the binary task rather than forced into a class.

2.3 Why anyone cares

Two honest motivations and one dishonest one.

- **Access to justice.** India’s courts carry a backlog measured in tens of millions of cases. A litigant without money cannot easily learn whether their claim resembles claims that have succeeded. A system that could say “here are the rules and the authorities that apply to facts like yours” would be useful even if it never predicted an outcome.
- **Scientific interest.** If a model can predict outcomes from pre-decision material alone, that tells us something real about how much of a legal outcome is determined by the facts. If it cannot, that is equally informative.
- **The dishonest one.** “Our system predicts court decisions with 78% accuracy” is a headline. It is also, as this project demonstrates, substantially an artifact of what the model is allowed to read.

2.4 What was already published

The reference point for this project is **ILDC/CJPE** (Malik et al., *ACL 2021*), which built a corpus of 34,816 Indian Supreme Court proceedings and reported a best accuracy of **78%**, against **94%** for human legal experts.

The ILDC authors are careful and explicit about the obvious trap. They state that the “end section(s) directly stating the decision have been deleted from the documents in ILDC since that is what we aim to predict.” So ILDC does *not* leak the operative order, and any claim that it does would be wrong. We checked this and confirmed it.

What ILDC *retains* is item (5) above — the court’s reasoning. The authors say so plainly: “we consider (along with the facts) the entire case (except the judgment).” Their best model reads the **last 512 tokens** of the document, chosen empirically and justified on the grounds that “the last parts of case proceedings usually contain the main information about the case and the rationale behind the judgment.”

That is a legitimate task definition. It is just a different one from the task a litigant faces. Measuring the difference is this project’s central contribution.

2.5 The two questions

The questions this project actually answers

Q1. How much of reported legal-judgment-prediction accuracy comes from the court’s reasoning rather than from the facts of the dispute?

Q2. Does converting facts into structured, symbolic legal objects — the thing an ontology-driven pipeline is *for* — help or hurt?

The answers, established in Section 6, are: **far less than it appears, because most of the apparent contribution is not reasoning at all but residual disposition language; and it hurts, substantially and measurably.**

3 The one idea: leakage

3.1 An analogy

Suppose you want to predict football results. You build a model, feed it match reports, and report 89% accuracy. Then someone notices that your match reports include the final score.

Nobody would defend that. But now suppose you carefully delete the score line, and keep everything else the report says — including the paragraph that reads “Liverpool’s defence had collapsed by the hour mark and the result was never in doubt thereafter.” You have removed the score. You have not removed the *knowledge* of the score. The report was written by someone who knew how it ended, and that knowledge is distributed through every sentence.

A court’s reasoning section is that paragraph. It is written after the decision, by the judge who made it, for the purpose of justifying it. The phrase “we are not persuaded by the appellant’s contention” is not evidence about the dispute; it is the outcome, restated.

Leakage is the general name for this: information in the model’s input that would not be available at prediction time in the real use case.

3.2 What must be removed, and why

The project’s masking rule (specified in `PROJECT.md` §5.2, implemented in `src/data/mask.py`) keeps only material that existed before the court decided:

Section	Kept?	Why
Caption, parties, judges	no	Judge identity is a real predictor but not a legal one; party names let a model memorise repeat litigants.
Facts	yes	This is the genuine input.
Procedural history	isolated	Which lower court decided what is <i>legitimately</i> known in advance, but it is also a shortcut, so it is placed in its own feature group (Q) and never mixed in silently.
Arguments of counsel	yes	Written by the parties, before the decision.
Court’s reasoning	no	Written after, knowing the answer.
Operative order	no	This is the label.
Headnote (pre-2000)	no	An editorial summary added by the law reporter that states both the outcome and the authorities outright.
Page furniture	no	“Indian Kanoon — http://indiankanoon.org/doc/... ”, page numbers, running headers.

After masking, the average case retains about 2,500 words of the original 5,300.

3.3 The control that makes everything else interpretable

Here is the problem with claiming you removed the leakage: how would you know?

The answer is a **sensitivity control**. We build a deliberately leaky arm — a model trained on *only* the text we deleted, the operative order — and check that it scores near-perfectly. If it does not, our detector of “where the order is” is broken, and the masked arm cannot be trusted either.

Input	Accuracy	Macro-F1	AUROC
Majority class (always predict LOSE)	0.528	0.345	0.500
Prior court’s disposition only	0.562	0.562	0.585
Masked text (facts + arguments)	0.629	0.626	0.674
Whole unmasked judgment	0.746	0.745	0.827
The removed order only (control)	0.931	0.931	0.981

Table 1: The leakage probe, forum_heldout split, $n_{\text{test}} = 902$. Source: experiments/paper3/leakage_probe_forum_heldout.json.

The last row is the point. At AUROC 0.981 the deleted text is almost perfectly predictive, which confirms two things at once: our order-detector finds the right text, and removing it removes something that mattered enormously. The gap between 0.827 (whole judgment) and 0.674 (masked) is the measure of how much the court’s own writing was doing.

A second version of the same check uses a large language model instead of a classifier, and agrees: shown the order alone it reaches 95.7% accuracy; shown the masked text, 62.0% (experiments/paper3/llm_probe_forum_heldout.json, $n = 350$).

Why this matters more than the headline number

Any paper reporting accuracy on this task should report a sensitivity control. Without one, a reader cannot distinguish “we removed the leakage” from “we believe we removed the leakage”. This project’s masked arm is worth reading precisely because the control next to it is near 1.0.

4 The data

4.1 The corpus

All 26,688 Indian Supreme Court judgment PDFs from 1950 to 2025 are held locally (6.8 GB). Of these, a provided spreadsheet identifies **6,954** as land or property disputes, and that subset is the working corpus for everything reported here.

Resolving spreadsheet rows to PDF files sounds trivial and was not. An earlier approach joined on *normalised titles* and quietly achieved a 0.6% match rate, because judgment file-names carry a copy marker after the year (`. . . _on_23_May_1957_1`) while titles do not — so a normaliser that strips “trailing digits” from both removes the *year* from every title, and every pair then mismatches by exactly that year. The current adapter joins on the exact filename supplied by the spreadsheet and achieves **6,954 of 6,954 (100%)**, with the join rate asserted at import time (`MIN_PDF_JOIN_RATE = 0.99`) so that a regression stops the pipeline instead of producing a smaller, silently biased dataset.

Text extraction: 6,954 of 6,954 succeeded, with zero failures and zero near-empty results.

4.2 Where the labels come from

Nobody hand-labelled 6,954 outcomes. Instead two independent labellers were built and made to agree:

1. **A deterministic rule engine** (`src/data/labels.py`) that finds the operative order by regular expression and classifies it. This is harder than it reads. Indian orders routinely

dispose of several proceedings at once — “the appeal is allowed . . . and the writ petition stands dismissed” — and a naive matcher reads the second clause as the outcome. The fix was to rank proceedings into tiers, so that an *appeal* or *special leave petition* outranks a consequential *writ petition* or *application*. That change alone reduced rules-versus-LLM conflicts from 196 to 107.

2. A **language model** shown the same order window and asked for the same decision, which must quote the span it relied on.

Agreement: 87.0% on the 4,317 cases where both decided and the quote verified. Where they conflict, the case is marked UNRESOLVED and queued for human review rather than resolved by coin-flip or by trusting one labeller. There are 562 such cases.

Label	Cases	Share
LOSE	2,964	42.6%
WIN	2,365	34.0%
PARTIAL	624	9.0%
UNRESOLVED	562	8.1%
UNKNOWN	289	4.2%
REMAND	116	1.7%
OTHER	34	0.5%
Total	6,954	

Table 2: Merged outcome labels. The binary task uses WIN and LOSE only: 5,329 cases.

4.3 How the data is split, and why it is not random

A random train/test split would be wrong here for two separate reasons.

Reason one: groups. The same dispute often generates several judgments — an appeal, a review, a connected matter. If one lands in training and another in test, the model can memorise the dispute rather than learn anything. So cases are grouped and a group is never split.

Reason two: time. A model that is trained on 2020 cases and tested on 1970 cases is being allowed to use the future to predict the past. Real deployment never has that.

The project uses two splits, deliberately, because each tests something different:

- **forum_heldout** — three forums (courts of origin) are held out entirely. Tests generalisation to an unseen procedural context. Train 3,350 / dev 455 / test 902.
- **temporal_2004_2013** — train on everything up to 2004, develop on 2004–2013, test after 2013. Tests generalisation forward in time. Train 3,446 / dev 480 / test 768.

Both splits carry a hash of their contents, which every results file records, so a number can always be traced to the exact split that produced it.

Building the temporal split exposed a conflict between the two requirements above, and a unit test caught it. Grouping by a case group’s *maximum* year put 1970 cases into the “test after 2013” bucket; grouping by the *minimum* year put 2025 cases into “train up to 2004”. Both are time-travel. There is no assignment that satisfies both constraints for a group that straddles a boundary, so the 13 straddling groups are dropped and the count is recorded.

5 What was built

The system is a chain of stages, each writing a file the next one reads. Everything is resumable and everything is deterministic given its inputs.

5.1 Infrastructure: one door to the model

All language-model calls go through `src/llm/client.py`, which talks to ASU’s own hosted model endpoint (Voyager). Four decisions in that file matter more than they look:

- **A content-addressed cache.** Every call is keyed on `sha256(model, messages, parameters)` in SQLite. Re-running an experiment costs nothing — the LLM baseline experiment reported later ran at a 100% cache hit rate over 600 calls. The cache currently holds 162,230 entries.
- **“No answer” is distinct from “empty answer”.** A result carries `ok=False` when the call failed, which is *not* the same as a model that replied with nothing. Conflating the two is how a network outage becomes a scientific finding, which happened twice on this project before the distinction was enforced (Section 7).
- **A 90-second timeout.** The client library’s default is 600 seconds with three retries, which froze an early batch for over twenty minutes with no output.
- **Parallelism.** Extraction runs 96 workers, embeddings 24; throughput went from 10.3 to 38.6 documents per second.

5.2 Segmentation

Before anything can be extracted, a judgment must be cut into units. The first implementation split on blank lines and returned *one unit* for 57.7% of cases, because text extracted from these PDFs frequently contains no blank lines at all. A third tier — sliding sentence windows — was added. Median units per case went from 1 to 47, and the share of unusable cases from 57.7% to 2.6%.

5.3 Fact extraction, with verbatim grounding

A model reads each unit and proposes *facts*: atomic factual assertions, each of which must include a **verbatim quote** from the source text, plus who asserted it and whether it was disputed. The quote is then located in the original by `src/grounding.py`; a fact whose quote cannot be found is **discarded, not corrected**.

This is the project’s main defence against fabrication. The current **grounding rate is 0.92**: 92% of proposed facts carry a quote that is genuinely present in the judgment.

Getting there required fixing the *verifier*, not the extractor. An early version rejected 42% of quotes that were in fact correct, for two reasons: PDF text extraction inserts hyphenation artifacts (plain-`tiff`’s), and a 20-character minimum rejected short true quotes such as “Appeal dismissed.” (15 characters). After repairing hyphenation and lowering the floor, the verified rate on a hand-checked set went from 58.3% to 91.7% — with 15 of 16 quotes confirmed present by eye.

Choosing the extraction model. Six models were benchmarked on an identical 20-case set:

Model	Grounding	Leak	Attribution	Facts/case	s/case
llama4-scout-17b	0.92	0.03	0.27	18.4	4.2
llama4-maverick-17b	0.93	0.01	0.41	23.5	28.7
qwen3-30b-a3b-instruct-2507	0.92	0.03	0.40	10.5	15.7
olmo3-32b-instruct	excluded — ~50 s per call				
glm-5-3-flash	excluded — hangs on long prompts				
glm-5-3	excluded — too slow at corpus scale				

Table 3: Extractor benchmark, 20 cases stratified 1950–2022. Source: experiments/paper1/extractor_benchmark.json.

maverick is better on every quality axis and $6.8\times$ slower. scout was chosen for corpus-scale runs; the benchmark is kept so the trade-off is on the record rather than implicit.

Scale. The current fact layer holds **96,623 facts across 4,702 cases**, median 17 per case.

5.4 Claims, defences, elements

A second extraction pass identifies what each party is actually *asking for* — the claim (e.g. adverse possession, specific performance, an injunction) — and what is raised against it. 15,377 claim records exist.

The project’s ontology (ontology/ontology_v1.yaml, converted from a supplied Word document by src/data/ontology_convert.py) defines 15 claims, 66 elements, 6 defence families and 4 remedy tiers. An *element* is a legal requirement a claim must satisfy: adverse possession, for instance, requires possession that is open, continuous, hostile and for the statutory period.

5.5 The canonical vocabulary

Each extracted fact is then *canonicalised*: mapped from free text to a controlled label drawn from a 345-entry vocabulary generated from the ontology’s own named fields. “The plaintiff had been cultivating the land since 1962” becomes something like possession.continuous_since_date.

This step is the one Section 6.3 shows to be destructive, so it is worth being precise about what it does and why anyone would want it. The appeal of a controlled vocabulary is that it makes facts *comparable across cases* — you can count how often an element is satisfied, mine patterns, and reason symbolically. The cost is that two facts mapping to the same label become indistinguishable.

5.6 Claim families and fact patterns

With facts as symbols, the specification called for grouping claims into families and then mining *patterns* — co-occurring fact sets that predict outcomes within a family. Both were built (src/cluster/, src/patterns/), using HDBSCAN and agglomerative clustering for families and FP-Growth with closed itemsets for patterns, with chi-square significance testing, Benjamini–Hochberg correction for multiple comparisons, and bootstrap stability over 20 re-samples.

Section 6.3 reports what they found, which is nothing, and why that is itself a result.

5.7 Grounding facts in law: statutes and precedents

Two tasks that take facts as input and predict *legal authorities* rather than outcomes.

Statute prediction. Given the facts, predict which statutory provisions apply. This required building a citation normaliser (`src/authorities/normalize.py`) over a registry of 42 Acts, because real Indian citations arrive in three awkward forms: truncated by line breaks (the single most frequent “Act” in 1,200 sampled judgments is “Property Act” with 644 mentions — that is the *Transfer of Property Act* with its head cut off); as generic back-references (“the said Act”, 311+ mentions), resolvable only by looking back to the last Act actually named; and with implicit Acts (a bare “Article” means the Constitution). Coverage over 1,500 judgments: 58,295 citations, 61.4% resolved to a canonical code.

Precedent retrieval. Given the facts, retrieve the earlier cases the court will cite. Ten systems were compared, including BM25 (keyword), dense embeddings pooled three different ways, and reciprocal-rank fusion of both. Every retrieved authority is asserted to pre-date the query judgment — checked per result, not merely filtered for.

5.8 Outcome models and feature groups

The final predictor consumes the structured pipeline’s output as six explicitly separated feature groups, so that any claim about what drives a prediction can be tested by ablation:

Group	Contents	Features
F	canonical fact atoms	1,313
P	mined fact patterns	3
E	element satisfaction scores	0 (empty — see B2)
S	predicted statutes	49
R	retrieved precedents	6
C	case metadata (year, forum, ...)	38
Q	prior court’s disposition	6

Table 4: Feature groups, 1,415 features total. Q is isolated because it is a procedural shortcut rather than a legal signal.

5.9 Traces

For each prediction the system emits a *trace*: which facts, statutes and precedents it relied on. Section 6.6 describes the test that establishes these traces are faithful rather than decorative — which is not the same as establishing they are correct.

6 Results

6.1 Headline 1: the published result is mostly leakage

This is the project’s main finding, and it is not the finding the project expected — an earlier version of this document claimed the court’s *reasoning* was the source of the gap, and Section 6.2 reports the measurement that refuted it. Six input constructions were run on **identical**

cases, with an identical split, identical labels and an identical classifier (TF-IDF features into logistic regression). The *only* thing that varies between rows is which part of the judgment the model is allowed to read.

Input construction	Words	Accuracy	Macro-F1	AUROC
order_only (<i>sensitivity control</i>)	427	0.921	0.920	0.976
full (<i>whole judgment, ceiling</i>)	5,302	0.720	0.718	0.788
ildc_style (<i>all but the order</i>)	5,051	0.680	0.678	0.734
ildc_style_no_headnote	4,481	0.678	0.676	0.735
last_512_tokens (<i>ILDC's best input</i>)	511	0.776	0.775	0.845
masked (<i>facts + arguments only</i>)	2,501	0.621	0.619	0.668

Table 5: The reasoning ablation. Same 895 test cases, same 3,324 training cases, same classifier throughout. Source: experiments/paper1/reasoning_ablation_forum_heldout.json.

Paired bootstrap over the test cases (2,000 resamples), reporting the difference in AUROC with a 95% confidence interval:

Comparison	Δ AUROC	95% CI	Significant
last_512_tokens – masked	+0.1764	[+0.1480, +0.2064]	yes
ildc_style_no_headnote – masked	+0.0670	[+0.0535, +0.0820]	yes
ildc_style – masked	+0.0660	[+0.0539, +0.0789]	yes
full – ildc_style	+0.0538	[+0.0476, +0.0602]	yes

What this appears to say, and why that reading is wrong

Read naively, the table says the court’s reasoning is worth a great deal. Take *order_only* as the ceiling on what is knowable here (AUROC 0.976, so 0.476 of signal is available above chance); then the facts and arguments appear to carry $(0.668 - 0.5)/0.476 = 35\%$ of it, the whole order-removed document 49%, and the last 512 tokens 73%.

That reading has one weak point, and it turns out to be fatal to it. **The final 512 tokens sit exactly where the operative order used to be.** If the boundary between “reasoning” and “order” is even slightly imperfect, that arm — and no other — is contaminated. The next section measures whether it is.

Three objections, and what answers each

This is the kind of result that invites immediate objections. Each was tested; the second one won.

“Your masked arm is worse simply because it has less text”. No — the evidence runs hard the other way. The *best* non-leaky arm, *last_512_tokens*, reads **511 words**; the masked arm reads **2,501**. The winning construction has roughly one-fifth the text and beats the masked arm by 0.176 AUROC. Volume is not the mechanism; *which* text is.

“The last 512 tokens must still contain the order”. This objection is correct, and Section 6.2 is the measurement that establishes it. An earlier version of this document argued the objection was unlikely, on two indirect grounds: that the order detector is independently validated (the `order_only` arm reaches 0.976, so it finds the order rather than missing it), and that `ilddc_style` beats masked without any positional advantage. Both of those facts are true and neither settles the question, because the order detector can mark the boundary correctly and still leave the sentence *before* it in place. That is exactly what happens.

“This is just your corpus”. Correct, and this is the single most important qualification on the result. Everything above is measured on *our* 6,954 land-dispute cases with *our* masking, as a comparable construction of ILDC’s task — not on ILDC itself. The replication on ILDC’s own released text and labels is written (`src/eval/ilddc_ablation.py`, four arms including a positional control) and blocked only on dataset licence acceptance, which is the user’s to give and which this project deliberately did not route around.

One number worth keeping

The `last_512_tokens` arm reaches **accuracy 0.776**; ILDC’s published best is **78%**. A plain TF-IDF model, on a different and much smaller corpus, lands within half a percentage point of a published transformer result once it reads the same window the published model reads. That is worth keeping — but after the next section it is no longer a statement about reasoning. It is a statement about how much can be recovered from a window that both setups leave contaminated.

6.2 Where the advantage actually comes from

The claim to be tested is that `last_512_tokens` wins because the final window retains disposition language, not because the court’s reasoning is informative. Three measurements settle it, on 842 test cases (60 of the 902 were dropped for being shorter than 2,048 words in the order-removed text, which the setback arms require). **Every arm here is exactly 512 words by construction**, so volume cannot explain any difference between them.

1. The setback curve

Take four non-overlapping 512-word windows, stepping backwards from the end of the order-removed document. The two hypotheses predict different shapes, and both predictions were written into the module before it was run:

- **Residual leakage** — contamination lives next to the boundary and cannot reach back 512 words, so the curve should *fall off a cliff* after the first window.
- **Reasoning** — a court’s analysis occupies the last several thousand words, so signal should *decay gradually* with distance, with every window still above masked.

Arm	Window	AUROC	Δ vs masked	95% CI	Sig.
masked	facts + arguments (2,616 w)	0.656	—	—	—
w0	[−512:]	0.838	+0.1814	[+0.1501, +0.2134]	yes
w1	[−1024: − 512]	0.669	+0.0128	[−0.0158, +0.0410]	no
w2	[−1536: − 1024]	0.647	−0.0094	[−0.0358, +0.0154]	no
w3	[−2048: − 1536]	0.638	−0.0183	[−0.0457, +0.0070]	no

Table 6: The setback curve. Source: experiments/paper1/leakage_setback_forum_heldout.json.

This is unambiguously the leakage shape: a cliff, not a decay. **512 words of the court’s reasoning, taken from anywhere other than the very end, carry no more outcome signal than the facts do.**

2. A cue audit, against the right base rate

Why that shape? Run the disposition patterns from `src/data/labels.py` — the *same* regular expressions that produced this project’s outcome labels — over each window and count the cases containing an explicit disposition statement.

A raw count would be uninterpretable. Masked text is full of sentences like “the High Court dismissed the suit”, which recite what a *lower* court did and leak nothing about this court’s decision; the patterns match those too. So masked supplies the base rate, and only an *excess* over it is evidence of contamination.

Arm	Cases with a cue	Rate	Excess over base
masked	227 / 842	0.270	— <i>base rate</i>
w0	353 / 842	0.419	+0.150
w1	75 / 842	0.089	−0.180
w2	70 / 842	0.083	−0.186
w3	62 / 842	0.074	−0.196

Only the final window is *enriched* in disposition language. Every setback window sits well *below* the base rate — cleaner than masked — and scores the same as it.

The mechanism, visible in the text. Indian judgments announce the conclusion one sentence before the operative order:

“For the foregoing reasons we find no merit in the appeal. *The appeal is dismissed with costs.*”

The order detector marks the boundary at the second sentence, which is **correct** — that is the operative order. The first sentence stays, and it states the outcome just as plainly.

3. How deep the contamination reaches

The setback curve answers the question at 512-word granularity. Sweeping the same 512-word window backwards in fine steps locates the boundary precisely — and AUROC and cue rate fall together, which is what a single shared cause looks like:

Offset	AUROC	Cue rate	Δ vs masked	Sig.
0 words	0.838	0.419	+0.1814	yes
32 words	0.780	0.296	+0.1240	yes
64 words	0.719	0.208	+0.0625	yes
128 words	0.684	0.131	+0.0278	no
256 words	0.676	0.099	+0.0200	no
512 words	0.669	0.089	+0.0128	no

Table 7: Contamination depth. The advantage is gone once the window is set back **128 words**.

Deleting the operative order leaves roughly 100 words of outcome-telegraphing text behind, and that residue accounts for essentially all of the last-512 arm’s advantage.

Two subsidiary checks point the same way and bound how much a cue-detector alone can do. Deleting every cue-bearing *sentence* from w0 drops it from 0.838 to 0.775; evaluating the same model only on the 489 cases where no cue was detected gives 0.742, against masked’s 0.646 on those same cases. Both remain above masked, so sentence-level scrubbing does *not* fully clean the window — there is outcome-telegraphing language the patterns miss (“we find no merit in the submission”, “the conclusion is inescapable”). This is exactly why the setback curve is the primary evidence: it never relies on detecting a cue at all.

So does the reasoning add anything? Yes, but little

One loose end remains. The whole-document arm *ildc_style* *contains* the contaminated tail, so its advantage might be the same residue diluted across 5,000 words. Cutting the tail off answers it:

Comparison	Δ AUROC	95% CI	Sig.
w0 (last 512) – masked	+0.1814	[+0.1501, +0.2134]	yes
ildc_style – masked	+0.0588	[+0.0458, +0.0722]	yes
ildc_style – ildc_minus128 (<i>the tail alone</i>)	+0.0322	[+0.0273, +0.0376]	yes
ildc_minus128 – masked (<i>reasoning, cleaned</i>)	+0.0266	[+0.0143 , +0.0391]	yes
ildc_minus512 – masked	+0.0215	[+0.0096, +0.0334]	yes

With the final 128 words removed, the court’s *entire* reasoning still beats facts-and-arguments by +0.027 AUROC, and the interval excludes zero. So the reasoning does carry real signal — it is simply far smaller than it appears. Of *ildc_style*’s +0.059 advantage, more than half (+0.032) is the contaminated tail.

The corrected headline

The input construction used by the published state of the art appears to beat a facts-only input by **+0.181 AUROC**. **About 85% of that is an artifact of imperfect order removal.** Deleting the operative order leaves roughly 100 words of outcome-telegraphing text in place; a 512-word window set back past that residue scores no better than the facts at all (+0.013, CI [−0.016, +0.041]).

With the contaminated tail removed, the court’s reasoning adds **+0.027 AUROC** ([**+0.014**, **+0.039**]) — real, statistically significant, and **one-seventh** of what it appears to add.

Why this is a better result than the one it replaces. The earlier claim — “the court’s reasoning carries more signal than the facts” — was a claim about legal text. This one is a claim about *benchmark construction*, it is more precisely quantified, and it is actionable: a benchmark built this way needs roughly 128 words of margin past the detected order boundary, not zero. It also explains a puzzle the earlier framing left open, namely why a 511-word window would beat a 2,501-word one so decisively when it supposedly contained only a subset of the same kind of reasoning.

One caveat on scope. These arms run on 842 cases rather than the 895 of Section 6.1, because the setback windows need documents of at least 2,048 words. The two tables are therefore not directly comparable row by row — masked is 0.656 here and 0.668 there — but every comparison *within* each table is paired on identical cases, which is what the confidence intervals are computed over.

6.3 Headline 2: turning facts into symbols destroys the signal

The second question was whether the structured, ontology-driven representation helps. To answer it cleanly, the project built a **representation ladder**: the same classifier, the same split, the same labels and the same 881 test cases at every rung, with *only the representation* changing.

Rung	Representation	Features	Macro-F1	AUROC
0	majority class	—	0.346	—
1	masked judgment text	50,000	0.608	0.657
2	extracted fact text	41,142	0.586	0.614
3	canonical fact atoms	1,313	0.505	0.517

Table 8: The representation ladder, `forum_heldout`, $n_{\text{test}} = 881$. Source: `experiments/paper1/representation_ladder_forum_heldout.json`.

Read down the AUROC column. Extraction costs **0.043**. Canonicalisation — converting those extracted facts into the controlled vocabulary — costs a further **0.097**, landing at 0.517, which is essentially chance. The pipeline’s symbolic output carries almost no information about the outcome, while the raw text it was derived from carries a measurable amount.

Five independent lines of evidence for the same conclusion

A single ladder could be a bug. Five separate tests were run and all agree.

1. **The ladder itself:** 0.657 → 0.614 → 0.517.
2. **Is it the *ontology*’s vocabulary that is bad?** No. An A/B test compared the authored ontology vocabulary against a vocabulary *induced* from the corpus at equal granularity. Raw text beats both decisively, and the two vocabularies are statistically indistinguishable from each other:

Comparison	ΔAUROC	95% CI	P
text — induced	+0.1138	[+0.0732, +0.1545]	0.000
text — ontology	+0.0887	[+0.0430, +0.1352]	0.000
induced — ontology	−0.0251	[−0.0711, +0.0200]	0.846

So the loss is not a defect of *this* vocabulary. It is a property of discretisation.

3. **Would more labels help?** No. A sweep from 345 to 1,000 vocabulary entries does not recover the gap, and a frequency-threshold sweep sits at chance at every threshold.
4. **Does the loss happen where the vocabulary fails, or where it succeeds?** This is the sharpest test (`src/eval/vocab_diagnosis.py`). Facts were divided into those the vocabulary *can* name and those it cannot, and the text-versus-atom comparison was run within each group:

Group	Text AUROC	Atom AUROC	Cost of naming
A. facts the vocabulary <i>can</i> name	0.544	0.457	0.087
B. facts it <i>cannot</i> name	0.526	0.524	0.002

The cost is concentrated *precisely where the vocabulary works*. That rules out the comfortable explanation (“coverage is just incomplete; fix the vocabulary”) and supports the uncomfortable one: successfully assigning a fact to a legal category is what discards the information. Note the small sample ($n_{\text{test}} = 143$ and 127) — group A’s interval spans zero at $P = 0.083$, so this line is suggestive rather than conclusive on its own.

5. **A pre-registered prediction.** Before running it, the project predicted that an LLM given canonical facts (LLM-CF) would do *worse* than the same LLM given raw text (LLM-0). It did: macro-F1 0.469 versus 0.491.

The downstream consequence: §8 finds nothing, correctly

If canonical atoms carry no signal, then pattern mining over canonical atoms must find nothing. It does, and the machinery was built well enough to say so rather than to manufacture a result:

Closed itemsets mined	498
Stable across $\geq 80\%$ of 20 bootstrap resamples	311
Patterns statistically tested	351
Pass uncorrected $p < 0.05$	10
<i>Expected to pass by chance alone</i>	17.6
Survive Benjamini–Hochberg correction	0

Ten patterns passed an uncorrected test — fewer than the 17.6 chance alone predicts. Without the multiple-comparisons correction, those ten would have shipped as a “pattern library”. This is the clearest example in the project of a control earning its place.

The claim taxonomy also fails to validate. Clustering 7,441 claims produced six clusters with a silhouette score of **0.0245** and 71.5% of points classed as outliers by HDBSCAN. A silhouette near zero means the clusters are not separated. When the clusters were merged into families with a minimum size, one family (PrivateTitlePossession) absorbed 2,365 cases and nearly every other label. The taxonomy is not supported by the data, and is reported as such rather than presented as a finding.

6.4 Where structured facts *do* pay off

The two results above are negative. They are not the whole story: on the tasks that predict *legal authorities* rather than outcomes, facts work well.

Statute prediction

Given only the facts, predict the statutory provisions that apply. 103 labels, $n_{\text{test}} = 796$.

The first version of this experiment produced micro-F1 0.447 and looked like a strong result. It was an artifact, and the thing that caught it was a **copy control**: a system that does nothing but copy provisions already mentioned verbatim in the input text scores **0.713** — far higher. The “prediction” was largely string matching.

The honest version removes from the target set every provision the input text mentions, leaving only provisions the model must genuinely infer. 46 labels, $n_{\text{test}} = 702$:

System	Micro-F1	P@1	R@10
Global top-5 prior	0.069	0.134	0.418
Claim-family prior (<i>strongest baseline</i>)	0.093	0.173	0.503
Copy from text (<i>control, now at floor</i>)	0.000	0.081	0.385
Masked text	0.173	0.236	0.660
Extracted fact text	0.159	0.206	0.581
Canonical atoms	0.109	0.161	0.464

Table 9: Statute prediction on *novel* labels only — provisions not mentioned in the input. Source: experiments/paper2/statute_prediction_forum_heldout_labels_novel.json.

Micro-F1 0.173 against the strongest baseline’s 0.093 — 1.9×, with the copy control driven to zero by construction. This is a real positive result, and the same ordering appears again: raw text > extracted facts > canonical atoms.

Precedent retrieval

Given the facts, retrieve the earlier cases the court will actually cite. 150 queries. An important honesty constraint applies: only **1,035 of 2,113** cited authorities (49.0%) are themselves in this corpus, because Supreme Court judgments cite High Courts, the Privy Council and English decisions. Recall is reported against the *reachable* set with the raw figure stated beside it.

System	R@10	R@50	MRR	nDCG@10
Mention baseline	0.022	0.112	0.037	0.014
Popularity baseline (<i>control</i>)	0.032	0.103	0.072	0.032
Canonical atom overlap	0.021	0.058	0.046	0.021
Dense, mean-pooled	0.053	0.130	0.114	0.054
Dense, max-pooled	0.039	0.104	0.097	0.044
BM25 (keyword)	0.207	0.350	0.431	0.225
Dense over masked text	0.229	0.390	0.380	0.221
RRF(BM25 + dense masked text)	0.238	0.453	0.428	0.243

Table 10: Precedent retrieval, 150 queries, recall against 1,035 reachable authorities. Source: experiments/paper2/precedent_retrieval_forum_heldout.json.

The fused system reaches **R@10 0.238 and MRR 0.431 against a popularity control of 0.032 and 0.072** — roughly **7.4×** the control on R@10. Because the control is popularity (always return the most-cited cases), beating it by that margin establishes the system is responding to the facts rather than exploiting citation skew.

Two sub-results worth noting. Canonical atom overlap (0.021) is *below* the popularity control — the symbolic representation fails on this task too. And max-pooling the dense representation made it worse (0.039 vs 0.053), which was a prediction this project made and had to retract: the deficit is in the facts, not in the pooling.

6.5 Outcome prediction hits a ceiling, and the ceiling looks real

Twenty-four outcome models were run on 881 test cases across 1,415 features. Selected rows:

Model	Accuracy	Macro-F1	95% CI	ECE
Majority (global)	0.530	0.346	[0.332, 0.360]	—
Majority (per decade)	0.571	0.557	[0.527, 0.589]	—
1r F (facts only)	0.501	0.499	[0.464, 0.530]	0.270
1r F+P+E+S+R+C	0.513	0.513	[0.479, 0.546]	0.285
1r S only (statutes)	0.599	0.599	[0.567, 0.629]	0.041
1r Q only (prior court)	0.525	0.509	[0.474, 0.540]	0.034
gbm C only (metadata)	0.583	0.582	[0.550, 0.613]	0.091
gbm F+P+E+S+R+C	0.598	0.597	[0.566, 0.630]	0.107

Table 11: Outcome models, $n_{\text{test}} = 881$. ECE is expected calibration error — how far the model’s stated confidence is from its observed accuracy; lower is better. Source: experiments/paper3/outcome_forum_heldout.json.

Three uncomfortable observations:

1. **Every confidence interval spans about 0.11 and they all overlap.** The best structured model (0.597) and a per-decade majority baseline (0.557) are not distinguishable at this sample size. Reporting the former as a result without the latter beside it would be misleading.
2. **Adding facts to metadata does not help.** gbm C only — year and forum, no facts at all — reaches 0.582. Facts alone reach 0.517. McNemar’s exact test confirms the metadata-and-authority groups genuinely beat the fact group ($P = 0.0008$ for logistic regression, $P = 0.004$ for gradient boosting), which is a real difference in the wrong direction for the project’s original thesis.
3. **Calibration and accuracy come apart.** 1r S only has the best calibration in the table (ECE 0.041) at competitive accuracy, while the best-accuracy model is poorly calibrated (0.107). For a system meant to tell a litigant how confident it is, the calibrated model may be the more useful one.

Large language models do no better

Five LLM configurations on the same 300-case subset, same labels:

System	Accuracy	Macro-F1	AUROC	ECE
LLM-0 (zero-shot)	0.577	0.491	0.579	0.265
LLM-FS (few-shot, $k = 4$)	0.543	0.522	0.522	0.117
LLM-CoT (chain of thought)	0.583	0.582	0.610	0.207
LLM-CF (canonical facts)	0.560	0.469	0.545	0.062
LLM-CF+A (canonical + authorities)	0.560	0.503	0.535	0.046
Structured gbm (same cases)	0.603	0.599	0.628	0.109
TF-IDF on masked text	0.597	0.593	0.645	0.045
Majority (per decade)	0.587	0.571	—	—

Table 12: Head-to-head, $n = 300$, llama4-scout-17b. Sources: experiments/paper3/llm_baselines_forum_heldout.json, head2head_forum_heldout.json.

Chain-of-thought’s apparent advantage over zero-shot is not significant (McNemar $P = 0.926$). Nothing in the table clears a per-decade majority baseline by a margin the sample size can resolve. The ceiling is not an artifact of choosing a weak model family.

Error analysis suggests the ceiling is intrinsic

354 errors on 881 cases were categorised, with a **base-rate control** that turns out to be decisive. For each diagnostic signal, compare the error rate among cases where the signal fires to the overall error rate of 0.402. The ratio is the *lift*; a lift of 1.0 means the signal tells you nothing.

Signal	Errors	$P(\text{error} \mid \text{flag})$	Lift
family_unassigned	39	0.448	1.12
statute_mismatch	175	0.398	0.99
discretionary	67	0.372	0.93
extraction_thin	25	0.352	0.88
atoms_only_generic	1	0.250	0.62

Table 13: Error-signal diagnosticity against a base error rate of 0.402. Source: experiments/paper3/error_analysis_forum_heldout.json.

Every lift is approximately 1.0. Not one of the pipeline’s own quality signals predicts where it will fail. Had these been reported as raw counts — “175 errors involve a statute mismatch” — they would have read as a diagnosis and an obvious work plan. Against the base rate they are noise.

The second test is stronger. If the model’s errors were caused by *pipeline* defects, they should differ from the errors of a completely different system. On the 300 shared cases the structured model made 119 errors and LLM-CoT made 125; under independence one expects $119 \times 125 / 300 = 49.6$ cases wrong for both. The observed number is **46**. The two systems’ errors are statistically independent — they are not failing on the same cases, and therefore not failing for a shared, fixable reason. The remaining difficulty looks intrinsic to predicting outcomes from pre-decision facts in this corpus.

6.6 The explanations are faithful

A system that outputs a prediction plus a list of “the facts I relied on” invites a simple question: did it rely on them, or is the list decoration?

The test is a **deletion test**. Delete the facts the trace cites, re-run the model, and measure how much the predicted probability moves. Compare against deleting the same number of random facts, and against deleting the facts the trace ranked *least* important.

One design decision matters here. The deletion is applied to the **facts**, and features are recomputed from scratch — not to the feature vector directly. Deleting features would be circular: of course removing a feature the model weighted heavily changes the output. Deleting the underlying facts tests whether the trace points at the right *evidence*.

Facts deleted	Mean Δ probability	vs cited	95% CI
Cited by the trace	0.0650	—	—
Random	0.0209	+0.0441	[+0.0245, +0.0659]
Inverse-ranked	0.0180	+0.0470	[+0.0291, +0.0666]

Table 14: Trace faithfulness, 45 cases. Cited facts move the prediction $3.1\times$ as much as random facts, winning on 77.8% of cases. Source: experiments/paper3/trace_faithfulness_forum_heldout_gbm.json.

Verdict: faithful. The facts the trace names are the facts the model used.

What faithfulness is not

Faithful does not mean *correct*. This test establishes that the explanation describes the model’s actual computation. It says nothing about whether that computation is legally sound. Given that the same model predicts outcomes near chance (Section 6.5), a faithful explanation of a near-chance prediction is an honest account of a weak inference — which is the right thing to produce, but should not be oversold.

7 What it took: the bugs that mattered

A reader evaluating this project should know that most of the effort went not into building the pipeline but into establishing that its numbers meant what they appeared to mean. Fifteen defects were found and fixed; each would have produced a confident, publishable, wrong answer.

Defect	What it would have caused	How it was caught
Segmentation returned one unit for 57.7% of cases (no blank lines in PDF text)	Extraction silently operating on 2.6% of its intended input	Median-units sanity check
Quote verifier rejected 42% of <i>correct</i> quotes (PDF hyphenation; 20-char floor)	Grounding rate understated; good extractions discarded	Hand audit of 16 quotes
Consequential dismissals read as the disposition	196 cases labelled with the wrong outcome	Auditing rules-vs-LLM conflicts
Group-whole and time-respect split constraints in direct conflict	Training on the future to predict the past	A unit test (tests/test_stage1.py)

Defect	What it would have caused	How it was caught
Fact attribution read only asserted_by, 84% court_narrative	Attribution coverage reported as 17% instead of 96%	Inspecting the field distribution
Canonicaliser’s self-reported “NEW” rate of 0.7%	A vocabulary declared complete while 16.7% of facts were un-nameable	Checking the answers instead of the flag
Batching the canonicaliser with a shared label menu	Label fidelity 34% → 22.7%, un-investigated	Per-batch-size sweep (4/8/16)
canonicalize_one silently dead after a prompt-ID change	The <i>fallback</i> path returning None for everything	Reading the validation path
Driver ran two pipeline stages concurrently; only the last exit code checked	A failed stage advancing the chain as if it had succeeded	Noticing a 26-minute orphaned process
macOS idle sleep mistaken for a hang (stalls of 78, 194, 232 min)	Three runs killed and restarted for no reason	<code>pmset -g log</code>
Burden annotation: 92% one class, one value never used	Legal metadata that was a constant, read downstream as fact	A degeneracy gate that <i>refuses</i> to save
Order-window detector swallowed whole short documents (59 cases)	A third of all unusable cases, cause unknown	Unusable-case audit
Caption detector ate argument text (For the Appellant; page footers)	545 of 5,255 units — ~1.2M words corpus-wide — deleted as “caption”	Reading what was being dropped
Loop-variable shadowing in the metrics dump	Metrics overwritten by a loop counter	Implausible output value
Per-item LLM loop with no retries (twice)	A network outage reported as “the corpus does not state these elements”	<code>ok=False</code> ≠ empty answer

7.1 Four transferable lessons

1. **A control is worth more than a method.** The copy control killed a statute result (0.447 → honest 0.173); the popularity control validated a precedent result; the base-rate control dissolved an entire error taxonomy; Benjamini–Hochberg prevented a pattern library from shipping. In every case the control changed the conclusion.
2. **Distinguish “no answer” from “the answer is nothing”.** This single confusion produced two separate false findings on this project. A loop that makes one call per item needs retries and must count failures separately from empty results.
3. **Assert your joins.** The PDF join and the fact-extraction QA gate both assert their success rate and abort below it. A pipeline that silently processes 12% of its input is worse than one that crashes.

4. **Elapsed time is not progress, and 0% CPU is not a hang.** Three runs were killed before anyone checked the power log.

8 Claims this project held and gave up

Research projects usually report what survived. This one kept a record of what did not, because the retractions are more informative than the confirmations.

Belief	What the measurement said
“The last-512 window is clean, so its +0.176 advantage shows the court’s <i>reasoning</i> carries the signal.”	Wrong, and this document said so before it was tested. The window is cue-enriched (+0.150 over the base rate) and the advantage vanishes once it is set back 128 words. Roughly 85% of it is residual disposition language; the reasoning’s real contribution is +0.027. See Section 6.2.
“Extraction preserves the signal in the text.”	At $n = 881$ it loses 0.043 AUROC. Canonicalisation loses a further 0.097.
“The ontology’s vocabulary is the problem; an induced one will do better.”	Indistinguishable: induced – ontology = -0.025 , $P = 0.846$. The loss is discretisation itself.
“The procedural shortcut (prior court’s disposition) dominates everything.”	Weak. Q only reaches AUROC 0.498 at $n = 881$ — chance.
“Max-pooling will fix the dense retrieval arm.”	Made it worse: R@10 0.039 vs 0.053 mean-pooled. The deficit is in the facts.
“ILDC leaks the operative order.”	It does not. The authors explicitly delete it, and we verified the claim. What ILDC retains is the <i>reasoning</i> , which is a different and defensible choice.
“Recovering the 1.2M words of wrongly-deleted argument text will improve fact attribution.”	Flat: 0.20 $\rightarrow$ 0.19. Absolute attributed facts rose $\sim 15\%$ (15,899 $\rightarrow$ 18,358) but the <i>rate</i> did not move. The cause of the 20% attribution ceiling is still unknown.
“Swapping in a better grounding model will fix the missing Fact and Issue concepts.”	Tested across five models. Rules learned under one model fire on nothing under another, and the recovered rules were single-example artifacts that generalised to zero real sentences.
“A bigger model will produce usable element-burden metadata.”	Two models failed the same degeneracy gate; the larger, more instruction-disciplined one failed too.

The last two illustrate a pattern worth naming: on this project, *model capability was almost never the binding constraint*. Instruction discipline, evaluation design and data coverage were.

9 What is *not* established

This section exists so that no reader has to discover these limits for themselves.

9.1 Open blockers

B2 — element burden metadata does not exist. The ontology requires every element to record who bears the burden of proof, to what standard, and when it shifts. It supplies this for **0 of 66** elements. Two models were asked and both produced degenerate output (92% and 87% assigned to a single class; one standard never used at all, including where it unambiguously applies). A gate refuses to save such annotations, so feature group E is **empty** and every result labelled F+P+E+... contains no E. The fix is a few hours of expert hand annotation, not a larger model.

B3 — the vocabulary cannot be frozen. Measured over 79,496 canonical facts, **20.3% are out of vocabulary** against a 5% threshold for freezing. The seed vocabulary of 345 subcategories covers roughly three quarters of what real judgments state. Note that Section 6.3 shows improving this coverage would not rescue the predictive result — the cost of naming is concentrated where the vocabulary already succeeds.

B4 — there is no gold annotation set. The specification asks for at least 150 cases annotated by a legal reviewer with inter-annotator agreement measured on a 30-case overlap. The **150 cases are prepared** (Data/gold/gold_tasks.jsonl) and an annotation tool is built (src/annotate/app.py), but no reviewer has annotated them. Consequently **extraction precision and recall against gold cannot be computed**. Four proxies are in use, and it is worth being exact about what each does and does not establish:

Proxy	What it establishes
Quote grounding rate (0.92)	Bounds <i>fabrication</i> . Says nothing about recall.
Legal-statement filter	Bounds one <i>precision</i> failure mode.
Rules-vs-LLM agreement (87.0%)	Estimates <i>outcome-label</i> noise, not fact quality.
The representation ladder	Measures <i>downstream utility</i> — what we actually care about — but cannot attribute a loss to any specific extraction error.

This is the single biggest gap for a reviewer, and it is blocked on human time, not compute.

9.2 Other open items

- **The ILDC replication is not done.** Everything in Section 6.1 is measured on our corpus as a comparable construction of ILDC’s task. The module that runs the same ablation on ILDC’s own released text and labels is written and tested; the dataset is gated behind a licence whose acceptance belongs to the user, and the module refuses to route around the gate. Until it runs, the headline is a strong internally-controlled result, not a replication.
- **What the cue scrub cannot separate.** The residual-leakage audit is done (Section 6.2) and found contamination reaching about 128 words. But `w0_scrubbed` and `w0_cue_free` both remain above `masked`, so some outcome-telegraphing language in the final window escapes the patterns. Whether that remainder is undetected leakage or genuine summative reasoning is **not** resolved — the setback curve sidesteps the question rather than answering it.
- **Human verification of outcome labels.** The specification’s check of 200 labels is outstanding; the 562 UNRESOLVED conflicts are where it should start.

- **The attribution ceiling.** 19–20% of facts are attributed to a party. The hypothesis that missing argument text caused this was tested and refuted. The real cause is unknown.
- **Every outcome-model confidence interval spans about 0.11.** At $n = 881$ the structured model and a per-decade majority baseline cannot be separated. Larger n or a different task is required to say more.
- **§8 as specified is mis-specified for this corpus.** Pattern mining over canonical atoms cannot work when canonical atoms carry no signal. This is reported as a finding rather than quietly dropped, but it does mean a specified deliverable is vacuous rather than met.

10 Status of every number in this document

A fact-layer defect was fixed on 30 September 2026: the caption detector had been deleting argument text (row 13 of the table in Section 7). Fixing it changed the fact layer substantially — 79,496 facts → **96,623**; 4,581 cases → 4,702; median 13 per case → 17. Everything downstream of facts is therefore being recomputed, and this section says plainly which numbers are current.

Result	Status	Why
Leakage probe (§3.3)	current	TF-IDF over text; independent of the fact layer
Reasoning ablation (§6.1)	current	Re-run after the fix
Setback / contamination (§6.2)	current	New, 1 Oct 2026; TF-IDF over text
Corpus, labels, splits (§4)	current	Upstream of facts
Extractor benchmark (§5)	current	Fixed 20-case set
Fact counts, grounding (§5)	current	Post-fix: 96,623 facts, 0 dropped, QA 4,026/4,026 spans exact
Representation ladder rung 1	current	Text-based rung
Representation ladder rungs 2–3	<i>recomputing</i>	Derived from facts
Vocabulary A/B and diagnosis	<i>recomputing</i>	Derived from facts
Statute and precedent results	<i>recomputing</i>	Fact-derived arms; text arms unaffected
Outcome models, LLM baselines	<i>recomputing</i>	Feature groups F,P change
Patterns, claim families	<i>recomputing</i>	Derived from facts
Trace faithfulness, error analysis	<i>recomputing</i>	Derived from the outcome model

What the re-run cannot change. The two headline findings do not depend on the fact layer. Section 6.1 is TF-IDF over text throughout. Section 6.3’s *direction* is structural — a 1,313-feature discretisation of a 50,000-feature text signal cannot overtake it by gaining 21% more facts — though the exact magnitudes will move. The honest expectation is that the ladder gaps narrow slightly and the conclusion stands; if it does not, that will be reported.

At the time of writing, canonicalisation of the new fact layer is 77% complete (74,485 of 96,623).

11 Where the project stands

11.1 What exists and works

- A corpus of 6,954 land-dispute judgments, 100% resolved to source PDFs, fully text-extracted.
- Dual-labeller outcome labels at 87.0% agreement, with conflicts preserved rather than resolved.
- A masking implementation with a sensitivity control at AUROC 0.981 — the apparatus that makes every other number interpretable.
- Two principled splits, hashed, with group and time constraints enforced and tested.
- A fact extraction layer of 96,623 verbatim-grounded facts at a 0.92 grounding rate.
- A statute normaliser over 42 Acts handling truncation, back-references and implicit Acts.
- Working statute prediction ($1.9\times$ baseline) and precedent retrieval ($7.4\times$ control).
- A faithfulness-tested trace mechanism.
- A reproducible driver (`run_downstream.sh`) with per-stage abort-on-failure and a QA gate.

11.2 What the paper would say

The defensible contribution is not a system that predicts outcomes well — it does not. It is a measurement:

The contribution

1. On Indian Supreme Court land disputes, the input construction used by the published state of the art beats a facts-only input by $+0.181$ AUROC — and **about 85% of that gap is an artifact of imperfect order removal**, not the court's reasoning. Deleting the operative order leaves roughly **128 words** of outcome-telegraphing text behind; a 512-word window set back past it scores no better than the facts ($+0.013$, CI $[-0.016, +0.041]$). With the tail removed, the reasoning's real contribution is $+0.027$ AUROC, CI $[+0.014, +0.039]$. The practical consequence for anyone building such a benchmark is a concrete margin, not a caution.
2. Projecting facts into a symbolic legal vocabulary **costs 0.097 AUROC**, which is most of the signal that survives extraction. This is a property of discretisation, not of the particular vocabulary: an induced vocabulary of equal granularity is statistically indistinguishable ($P = 0.846$), more labels do not help, and the cost is concentrated precisely where the vocabulary succeeds in naming a fact.
3. Facts *are* predictive of **legal authorities** — statutes at $1.9\times$ the strongest baseline, precedents at $7.4\times$ a popularity control — which is where an ontology-driven pipeline should be pointed instead.

Result 3 is what rescues the project from being purely negative, and it is also the professor's original framing: facts in, rules and authorities out.

11.3 What comes next, in priority order

1. **Replicate on ILDC itself.** The single step that would move this from a strong internally-controlled result to a replication. Written and waiting on a licence.
2. **Get the 150 gold cases annotated.** Removes the largest reviewer objection: there is currently no precision or recall figure for extraction, only proxies.
3. **Refresh all fact-derived results** once the driver chain completes, and update the reports.
4. **Hand-annotate the 66 element burdens (B2)**, which unblocks feature group E and the element-level reasoning the ontology was built for.

12 Glossary

AUROC

Area under the receiver-operating-characteristic curve. The probability that the model scores a randomly chosen WIN case above a randomly chosen LOSE case. 0.5 is chance; 1.0 is perfect. Preferred here over accuracy because it does not depend on where the decision threshold is set.

Macro-F1

The F1 score computed per class and then averaged, so that doing well on the majority class alone cannot carry the number. A model that always predicts LOSE gets 53% accuracy but only 0.346 macro-F1.

ECE (expected calibration error)

How far a model's stated confidence sits from its observed accuracy. A model that says "70% confident" and is right 70% of the time has ECE near 0.

Paired bootstrap

To compare two systems, resample the test cases with replacement 2,000 times, recompute both scores on each resample, and look at the distribution of the *difference*. If the 95% interval excludes zero, the difference is unlikely to be sampling noise. "Paired" means both systems are scored on the same resample, which removes case difficulty as a confound.

McNemar's test

An exact test for whether two classifiers differ, which looks only at the cases where they disagree. Appropriate here because the two systems are evaluated on the same cases.

Benjamini-Hochberg correction

When 351 hypotheses are tested at $p < 0.05$, roughly 17 will pass by chance. This procedure adjusts the threshold to control the proportion of false discoveries among those reported. Applying it took this project's pattern library from 10 patterns to zero.

Lift (base-rate control)

The error rate among flagged cases divided by the overall error rate. A lift of 1.0 means the flag carries no information, however large its raw count.

TF-IDF

Term frequency $\times$ inverse document frequency: a representation of a document as weighted word counts, where words common across the whole corpus are down-weighted. Simple, fast, and in this project repeatedly competitive with much larger models.

BM25

A keyword-ranking function used by search engines. On this project's precedent-retrieval task it beat dense embeddings on long queries.

Dense retrieval

Representing documents as vectors from a neural embedding model and retrieving by vector similarity — as opposed to keyword matching.

RRF (reciprocal rank fusion)

A way of combining two ranked lists by summing the reciprocals of each item's rank in each list. Used here to fuse BM25 with dense retrieval, which beat both.

Closed itemset

In pattern mining, a set of co-occurring items that has no superset occurring equally often. Mining only closed itemsets avoids reporting thousands of redundant sub-patterns.

Leakage

Information in the model's input that would not be available when the prediction is actually needed. The organising concern of this entire project; see Section 3.

Sensitivity control

A deliberately leaky arm, included to prove the leakage detector works. If a model trained on the text you *removed* does not score highly, you did not remove what you thought you did.

Copy control

A baseline that does nothing but copy answers already present verbatim in the input. If it beats your system, your system is doing string matching. This one invalidated a statute result.

Popularity control

A baseline that always returns the most frequently cited items. Beating it shows a retrieval system is responding to the query rather than exploiting skew in the data.

Grounding

Requiring a model to quote its source verbatim, then verifying the quote appears in the original. An unverifiable quote means the fact is discarded.

Trace

The list of facts, statutes and precedents a prediction was based on.

Faithfulness

Whether a trace describes the model's actual computation. Distinct from, and much weaker than, correctness.

IRAC

Issue, Rule, Application, Conclusion — the standard structure of a legal analysis, and the output format the system's reports follow.

Adverse possession

A legal doctrine by which someone who openly occupies land that is not theirs, continuously and against the owner's interest, for a statutory period, may acquire title. Prominent in this corpus and used throughout as an example.

SLP (special leave petition)

The discretionary route by which most cases reach the Indian Supreme Court. Its disposition is the outcome label for a large share of this corpus.

A How to reproduce everything

1. **Environment.** `.venv/bin/pip install -r requirements.txt`. Create `.env` at the project root with `VOYAGER_API_KEY=<key>`, obtained from `voyager.rc.asu.edu` → LLM Access → Create Key. No VPN is required; the endpoint resolves to public addresses.

2. **Run the chain.** `./run_downstream.sh`. It re-executes itself under `caffeinate -dim` so macOS idle sleep cannot stall it, aborts on any stage’s non-zero exit, and runs a QA gate after extraction. Every stage is resumable.
3. **Headline experiments individually:**

```
python -m src.eval.leakage_probe -split forum_heldout
python -m src.eval.reasoning_ablation -split forum_heldout
python -m src.eval.leakage_setback -split forum_heldout
python -m src.eval.representation_ladder -split forum_heldout
python -m src.eval.vocab_ab -split forum_heldout
python -m src.authorities.statute_predict -split forum_heldout
    -rebuild-targets
python -m src.authorities.precedent_retrieve -split forum_heldout
python -m src.predict.models -split forum_heldout
python -m src.trace.evaluate -split forum_heldout -model gbm
```
4. **Every number in this document** is read from a file under `experiments/`. The figure generator (`src/report/figures.py`) reads the same files and *raises* rather than substituting a default if one is missing, so a stale figure cannot be produced silently.

B Where things live

Path	Contents
<code>PROJECT.md</code>	The specification. Section numbers (§5.2, §6.2, §10.1) refer to it.
<code>WORKLOG.md</code>	Chronological record of decisions, dead ends and bugs with reasoning.
<code>reports/</code>	Stage acceptance against the specification: M0–M5, BLOCKERS.md, ENV_NOTES.md.
<code>src/data/</code>	Adapter, labels, masking, segmentation, splits, ontology conversion.
<code>src/extract/</code>	Fact and claim extraction, canonicalisation, vocabulary.
<code>src/eval/</code>	The experiments: leakage probe, reasoning ablation, representation ladder, vocabulary A/B and diagnosis, error analysis, ILDC replication.
<code>src/authorities/</code>	Statute normalisation and prediction, precedent retrieval.
<code>src/predict/</code>	Feature construction, outcome models, LLM baselines.
<code>src/trace/</code>	Trace construction and the faithfulness deletion test.
<code>src/llm/client.py</code>	The single door to the model endpoint: cache, timeout, retries.
<code>src/grounding.py</code>	Verbatim quote location with hyphenation repair.
<code>experiments/</code>	Every result file. Nothing in this document is sourced elsewhere.
<code>Data/interim/</code>	Derived artifacts. Large ones are gitignored and rebuilt deterministically.
<code>Data/gold/</code>	The 150 prepared gold cases and the 66-element burden sheet (B2, B4).
<code>run_downstream.sh</code>	The driver.

Every figure in this document was read from a results file in `experiments/`. Where a number is awaiting recomputation, Section 10 says so.